Project 07 — RNN/LSTM Text Classification

Level: Basic | **Dataset:** IMDB Sentiment (HuggingFace) | **Framework:** PyTorch

---

Objective

Build an LSTM with an embedding layer from scratch for binary sentiment classification.
Cover: tokenization, vocabulary, nn.Embedding, LSTM mechanics, padding, packing.

Project Structure

```
07_lstm_sentiment/
├── notebooks/
│   ├── 01_eda_text.ipynb
│   ├── 02_tokenize_vocab.ipynb
│   └── 03_train_evaluate.ipynb
├── data/processed/
│   ├── vocab.pkl
│   └── processed_data.pkl
├── models/model.pkl
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Notebook 01 — EDA (`01_eda_text.ipynb`)

STOP 1 — Load Dataset

- `from datasets import load_dataset; ds = load_dataset("imdb")`
- 25,000 train, 25,000 test reviews, binary labels (0=neg, 1=pos)
- Print 3 sample reviews with their labels
- Print average/max/min review length (in words)
- **Agent stops here. Explain:**
  - What sentiment classification is
  - Why text needs different preprocessing than tabular data
  - What a "token" is (roughly: a word)
  - Why review length varies and why that's a challenge for neural nets (fixed input size needed)
- Wait for user confirmation before continuing

STOP 2 — Text Statistics

- Plot histogram of review lengths (in words)
- Find 95th percentile length → this becomes MAX_LEN
- Print most common and least common words
- **Agent stops here. Explain:**
  - Why we cap at 95th percentile and not max (outliers would waste memory)
  - What happens to reviews shorter than MAX_LEN (pad) and longer (truncate)
  - Why very short and very long reviews are often outliers
- Wait for confirmation

STOP 3 — Word Frequency Analysis

- Build a frequency dict of all words
- Print top 20 words: mostly stopwords ("the", "a", "is")
- Print bottom 20 words: rare typos and names
- **Agent stops here. Explain:**
  - Why we want to remove very rare words (min_freq threshold)
  - Why we might keep or remove stopwords for sentiment (unlike topic modeling)
  - What a vocabulary size of 10k-50k means
- Wait for confirmation

Notebook 02 — Tokenize & Vocabulary (`02_tokenize_vocab.ipynb`)

STOP 4 — Build Vocabulary

```python
from collections import Counter
counter = Counter()
for text in all_texts:
    counter.update(text.lower().split())
vocab = {'<PAD>': 0, '<UNK>': 1}
for word, freq in counter.most_common(MAX_VOCAB):
    if freq >= MIN_FREQ:
        vocab[word] = len(vocab)
```

- Use MAX_VOCAB=25000, MIN_FREQ=2
- Save vocab dict to `data/processed/vocab.pkl`
- **Agent stops here. Explain:**
  - What `<PAD>` token is: index 0, used to fill short sequences to MAX_LEN
  - What `<UNK>` token is: index 1, for words not in vocabulary
  - Why we sort by frequency (most common words get lower indices)
  - What "vocabulary" means in NLP vs everyday meaning
  - Trade-off: larger vocab = more memory but fewer UNKs
- Wait for confirmation

STOP 5 — Text to Indices

Write `encode(text, vocab, max_len)`:

- Lowercase, split by whitespace
- Map words to indices using vocab (default to UNK)
- Truncate to max_len or pad with 0 to max_len
- Return list of integers, length = max_len
- **Agent stops here. Explain:**
  - How text becomes numbers: "the cat" → [5, 22, 0, 0, ... 0]
  - Pre-truncate vs post-truncate: take first or last max_len tokens
  - Why post-truncate (keep ending) can be better for sentiment (conclusion at end)
  - What padding index=0 (PAD) means for the model
- Wait for confirmation

STOP 6 — Dataset & DataLoader

Write `IMDBDataset(Dataset)`:

- Store encoded sequences (already processed) and labels
- `__getitem__` returns (tensor, label)
- DataLoader with batch_size=64
- **Agent stops here. Explain:**
  - Why we pre-process all texts before training (not in **getitem**)
  - The memory tradeoff: store encoded sequences (faster) vs encode on-the-fly (less RAM)
  - For 25k reviews × 500 tokens = 12.5M integers = fine in RAM
- Wait for confirmation

Notebook 03 — Train & Evaluate (`03_train_evaluate.ipynb`)

STOP 7 — Embedding Layer

- `nn.Embedding(num_embeddings=VOCAB_SIZE, embedding_dim=128, padding_idx=0)`
- Inspect: what is the embedding weight shape? [VOCAB_SIZE, 128]
- Run one forward pass: input [B, SEQ_LEN] → output [B, SEQ_LEN, 128]
- **Agent stops here. Explain:**
  - What an embedding layer is: a lookup table (not a real matrix multiply)
  - Why `padding_idx=0`: gradients don't flow through PAD token
  - What embedding_dim=128 means: each word is a 128-dimensional vector
  - Why embeddings are learned (not fixed): words get semantically similar vectors during training
  - Relationship to Word2Vec: same idea, but trained with the task
- Wait for confirmation

STOP 8 — LSTM Architecture

```
nn.Embedding(VOCAB_SIZE, 128, padding_idx=0)
nn.LSTM(input_size=128, hidden_size=256, num_layers=2, batch_first=True,
        dropout=0.3, bidirectional=False)
nn.Dropout(0.5)
nn.Linear(256, 1)
nn.Sigmoid()
```

Forward:

```python
embedded = self.embedding(x)          # [B, L, 128]
out, (h_n, c_n) = self.lstm(embedded) # out:[B,L,256], h_n:[2,B,256]
last_hidden = h_n[-1]                 # [B, 256] — last layer, last timestep
out = self.dropout(last_hidden)
return self.fc(out)
```

- **Agent stops here. Explain:**
  - What LSTM hidden_size means: dimension of hidden state
  - What num_layers=2 means: stacked LSTMs
  - What batch_first=True means: [B, L, H] not [L, B, H]
  - Why we take h_n[-1] (last layer's final hidden state) not out[:,-1,:]
  - What h_n and c_n are: hidden state and cell state
  - The 4 gates of LSTM: forget, input, cell, output — at a conceptual level
- Wait for confirmation

STOP 9 — LSTM Cell State Deep Dive

- Draw/explain the LSTM cell diagram in markdown
- Implement a minimal LSTM cell from scratch to understand the math:

```
f = sigmoid(Wf @ [h,x] + bf)   # forget gate
i = sigmoid(Wi @ [h,x] + bi)   # input gate
g = tanh(Wg @ [h,x] + bg)      # candidate cell
o = sigmoid(Wo @ [h,x] + bo)   # output gate
c = f*c_prev + i*g              # new cell
h = o * tanh(c)                 # new hidden
```

- **Agent stops here. Explain:**
  - What the forget gate decides: how much of old cell state to keep
  - What the input gate controls: how much new info to add
  - Why tanh is used on cell state (keeps values in [-1,1])
  - Why LSTM solves vanishing gradient: cell state flows with only element-wise ops
  - Difference from RNN: RNN has only h_t, LSTM has h_t and c_t
- Wait for confirmation

STOP 10 — Padding & PackedSequence

- Show that PAD tokens in LSTM output are wasted computation
- Use `nn.utils.rnn.pack_padded_sequence` with actual lengths
- Use `pad_packed_sequence` to unpack
- Compare training speed and loss with vs without packing
- **Agent stops here. Explain:**
  - What PackedSequence does: skips PAD tokens in LSTM computation
  - Why we need actual sequence lengths to pack
  - When packing matters most (highly variable-length sequences)
  - For IMDB with fixed MAX_LEN truncation, packing helps less — but still good practice
- Wait for confirmation

STOP 11 — Training Loop

- 15 epochs, Adam lr=0.001, BCELoss
- Track train/val loss and accuracy
- Apply gradient clipping: `nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`
- **Agent stops here. Explain:**
  - Why gradient clipping is important for RNNs (exploding gradients)
  - What max_norm=1.0 does: if gradient norm > 1, scale all gradients down
  - Why RNNs suffer from exploding gradients more than feedforward nets
  - How BCELoss works with Sigmoid output (vs CrossEntropyLoss)
- Wait for confirmation

STOP 12 — Evaluation & Embedding Visualization

- Test accuracy, F1, confusion matrix
- Extract embedding weights: `model.embedding.weight.data` [VOCAB_SIZE, 128]
- Use PCA to reduce to 2D, plot top 200 most frequent words
- Color by simple sentiment polarity (good/bad words if known)
- **Agent stops here. Explain:**
  - What embedding visualization shows: semantically similar words cluster together
  - What dimensions in embedding space roughly correspond to
  - How learned embeddings differ from random ones (cluster structure)
- Wait for confirmation

STOP 13 — Save & Inference

- Save model.state_dict(), vocab dict
- Write `predict_sentiment(text)` → positive/negative + probability
- **Agent stops here. Explain:**
  - Complete inference pipeline: raw text → encode with same vocab → pad/truncate → [1, MAX_LEN] tensor → model → sigmoid → float
  - Why we must use the same vocab saved during training
  - What happens to words not in vocab at inference (mapped to UNK)
- Wait for confirmation

`dashboard_core.py`

Functions:

- `load_model_and_vocab()` → model, vocab
- `predict_sentiment(text)` → label, probability
- `get_training_curves()` → dict
- `get_embedding_viz()` → 2D coords + word labels for top 200 words

`app.py` — Streamlit (~80 lines)

Sections:

1. Text area: user types any movie review
2. "Analyze" button → show Positive/Negative label + probability gauge
3. Tab 1: Training loss + accuracy curves
4. Tab 2: Embedding PCA visualization (scatter plot, 200 words)
5. Tab 3: Confusion matrix + F1

Key Concepts Covered

- Vocabulary building (<PAD>, <UNK> tokens)
- nn.Embedding as lookup table with padding_idx
- LSTM gates (forget, input, cell, output)
- h_n vs output: why we use last hidden state
- Gradient clipping for RNN stability
- PackedSequence for variable-length sequences
- BCELoss with Sigmoid for binary classification
- Embedding visualization with PCA